Université Cadi Ayyad

École Supérieure de Technologie (EST)

El Kelâa des Sraghna

Filière DUT : Ingénierie des données

PARTIE RECHERCHE

Smart Data Pipeline Auto-Adaptatif pour la Gestion Intelligente des Flux de Données

---

Préparées par :

BOUDALLAA SAMAH

EL BAHLOUL CHAYMAE

EL BASRI KHADIJA

Encadré Par : Pr. Anas BERKA

Le 11 Janvier 2026

I. Introduction

Avec l'essor massif des données générées par les systèmes IoT, les applications cloud, les plateformes mobiles et les services en ligne, les organisations sont confrontées à un défi majeur [1] : gérer des flux de données volumineux, variés et hautement changeants. Les pipelines de données traditionnels, conçus pour un environnement stable et statique, montrent rapidement leurs limites lorsqu'il s'agit de s'adapter à l'évolution dynamique du volume, de la vitesse ou du format des données.

Dans ce contexte, l'émergence des *smart data pipelines*, ou pipelines intelligents, constitue une réponse prometteuse. Ces systèmes visent à traiter les données en continu tout en s'adaptant automatiquement aux variations du flux, minimisant ainsi les interventions humaines et augmentant l'efficacité opérationnelle.
Notre recherche s'inscrit dans cette dynamique et explore les approches existantes afin de concevoir un pipeline auto-adaptatif capable de gérer des flux complexes en toute autonomie.

II. Contexte général et problématique

2.1. Explosion des données modernes

Les systèmes modernes produisent des quantités colossales de données : capteurs IoT, logs applicatifs, plateformes e-commerce, systèmes de paiement, objets connectés, réseaux sociaux… Ces flux sont :

- ✓ Massifs (volume),
- ✓ Rapides (vitesse),
- ✓ Variés (formats),
- ✓ Et irréguliers (variabilité), communément décrits par le modèle des 5V du Big Data [2], largement repris dans les architectures modernes de gestion des données [15]

2.2. Limites des pipelines classiques

Les pipelines traditionnels présentent plusieurs faiblesses :

- ✓ **Non adaptatifs :** ils nécessitent des modifications manuelles en cas de changement de schéma ou de variation du débit.
- ✓ **Sensibles aux pannes :** un changement de format peut bloquer tout le pipeline.
- ✓ **Peu scalables :** ils ne réagissent pas automatiquement aux pics de charge.
- ✓ **Manque d'intelligence :** pas de prise de décision basée sur des métriques.

2.3. Problématique

Comment concevoir un **pipeline de données intelligent**, capable de :

- ✓ Détecter automatiquement les variations des flux,
- ✓ Prendre des décisions en continu,
- ✓ Ajuster son comportement sans intervention humaine ?

III. Concepts fondamentaux

3.1. Pipeline de données

Un pipeline de données représente l'ensemble des processus permettant d'ingérer, transformer, traiter et stocker des données provenant de sources diverses [3].

3.2. ETL vs ELT

- ✓ **ETL** : extraction → transformation → chargement [4]
- ✓ **ELT** : extraction → chargement → transformation
  Les pipelines modernes tendent vers ELT, plus adapté aux architectures distribuées [4].

3.3. Batch vs Streaming

- ✓ **Batch** : traitement par lots, utile pour les données historiques [5].
- ✓ **Streaming** : traitement en temps réel, indispensable pour les flux continus (IoT, logs, transactions) [5].

3.4. Data Lake et Lakehouse

- ✓ **Data Lake**: stockage brut, flexible [6].
- ✓ **Lakehouse** : combine la flexibilité du lake + performances du warehouse [6].

3.5. Concepts clés supplémentaires

- ✓ **Backpressure** : gestion de la pression dans les systèmes de streaming
- ✓ **Scalabilité** : capacité d'adaptation aux variations de charge
- ✓ **Observabilité** : logs, métriques, tableaux de bord
- ✓ **Schema Evolution** : gestion automatique des changements de schéma

IV. Architectures modernes pour la gestion des données

4.1. Architecture Lambda

Combine traitement batch + temps réel. Flexible mais complexe [7].

4.2. Architecture Kappa

100% streaming. Simple, moderne et adapté aux flux continus [8]. Cette architecture est aujourd'hui largement adoptée dans les pipelines temps réel industriels [15].

4.3. Architecture Lakehouse

Fusion du data lake avec un moteur transactionnel. Exemple : Delta Lake, Iceberg, Hudi [6].

4.4. Architecture événementielle

Basée sur Kafka ou Pulsar, elle repose sur des messages publiés en continu. Les architectures orientées événements sont considérées comme une base essentielle des smart data pipelines modernes [16].

4.5. Cloud-native

Déployée sur Kubernetes, scalable automatiquement. Cette approche est largement recommandée pour les pipelines auto-adaptatifs à grande échelle [17]

Ces architectures constituent les fondations des pipelines intelligents.

V. Approches d'adaptativité dans les pipelines de données

C'est le cœur scientifique de cette recherche.

5.1. Adaptation basée sur des règles

Le système prend des décisions selon des règles prédéfinies :

- ✓ Si le débit dépasse un seuil → augmenter la capacité,
- ✓ Si le schéma change → appliquer une conversion automatique,
- ✓ Si un nœud tombe → rerouter les données.

Cette approche est simple mais limitée [9]. Elle reste cependant très utilisée dans les systèmes industriels pour sa simplicité et sa fiabilité [18].

5.2. Adaptation basée sur le Machine Learning

Le pipeline analyse les métriques et prédit les actions idéales :

- ✓ Prédiction du débit futur,
- ✓ Détection d'anomalies,
- ✓ Anticipation des erreurs [10]. Plusieurs travaux industriels montrent l'efficacité du ML pour l'optimisation des pipelines de streaming [19].

5.3. Adaptation via l'Apprentissage par Renforcement

Le pipeline apprend à optimiser son propre comportement en continu :

- ✓ Auto-scaling basé sur récompenses,
- ✓ Ajustement dynamique des buffers,
- ✓ Amélioration automatique des performances [11]. Cette approche reste émergente mais prometteuse dans les pipelines auto-adaptatifs [20].

5.4. Adaptation basée sur les politiques

Décisions automatiques guidées par :

- ✓ SLA,
- ✓ SLO,
- ✓ Seuils dynamiques.

VI. Technologies utilisées dans les pipelines modernes

Voici les outils les plus utilisés aujourd'hui :

6.1. Ingestion

- ✓ Apache Kafka [12]
- ✓ Apache Pulsar
- ✓ MQTT

6.2. Traitement

- ✓ Apache Spark Streaming [13]
- ✓ Apache Flink
- ✓ Apache Beam

6.3. Orchestration

- ✓ Apache Airflow [14]
- ✓ Prefect
- ✓ Dagster

6.4. Stockage

- ✓ Delta Lake
- ✓ Apache Iceberg
- ✓ Apache Hudi

6.5. Monitoring

- ✓ Prometheus
- ✓ Grafana
- ✓ ELK Stack (Elastic, Logstash, Kibana)

VII. Analyse des projets similaires (étude comparative)

7.1. Projets open-source similaires

a) Smart Data Lake Builder

- ✓ Framework complet ETL/ELT
- ✓ Automatisation des transformations
- ✓ Limite : peu orienté auto-adaptation temps réel
- ✓ Lien : https://www.smartdatalake.io/

Ce type de pipeline est représentatif des architectures décrites dans la littérature technique récente [15].

b) End-to-End Data Pipeline (Kafka + Spark + Airflow)

- ✓ Architecture complète et modulable
- ✓ Utilisé pour les pipelines industriels
- ✓ Limite : nécessite une configuration manuelle des réactions à la charge
- ✓ Lien : https://github.com/giacbrd/SmartPipeline

c) SmartCity Real-Time Pipeline

- ✓ Conçu pour des données IoT
- ✓ Bon exemple de gestion de flux massifs
- ✓ Limite : pas d'intelligence d'adaptation avancée
- ✓ Lien : https://github.com/LiliValGo/SmartCity

7.2. Projets industriels

- ✓ Pipelines d'AWS, Google Cloud, Azure

Systèmes auto-scalables basés sur Kubernetes. Ces pipelines industriels s'appuient fortement sur des mécanismes décrits dans les plateformes de veille technologique [16]

- ✓ Monitoring intelligent basé sur ML

7.3. Synthèse comparative

Les projets existants proposent :

- ✓ Efficacité,
- ✓ Scalabilité,
- ✓ Automation partielle,

Mais présentent des limites sur :

- ✓ L'auto-adaptation complète,
- ✓ La compréhension sémantique des données,
- ✓ La gestion automatique des anomalies,
- ✓ La prise de décision autonome.

VIII. Gap Analysis : limites des solutions existantes

Les travaux actuels présentent plusieurs insuffisances :

- ✓ L'auto-adaptation se limite souvent au scaling, pas au contenu des données.
- ✓ Les changements de schéma sont rarement gérés automatiquement.
- ✓ Peu de solutions hybrides combinant règles + ML + RL.
- ✓ Manque de systèmes capables de décider eux-mêmes des actions optimales.
- ✓ Les pipelines actuels restent fortement dépendants des DevOps et Data Engineers. Cette limite est également soulignée dans plusieurs études industrielles récentes [18].

D'où la nécessité de développer un pipeline réellement intelligent.

L'analyse des architectures existantes, des approches d'adaptativité et des projets similaires met en évidence un besoin croissant de pipelines capables de dépasser l'automatisation partielle actuellement proposée. En particulier, la majorité des solutions étudiées reste dépendante de configurations statiques ou d'interventions humaines. Ces constats motivent la proposition d'un pipeline de données intelligent et auto-adaptatif, présentée dans la section suivante.

Cette proposition correspond à une architecture conceptuelle qui servira de base à l'implémentation du prototype dans la suite du projet de fin d'études.

IX. Proposition : contribution de notre projet

Notre PFE se donne pour objectif de développer :

Un Smart Data Pipeline Auto-Adaptatif capable de :

- ✓ Détecter automatiquement les variations du flux,
- ✓ S'adapter en temps réel (scaling, mapping, transformation),
- ✓ Optimiser les performances sans intervention humaine,
- ✓ Fournir une observabilité complète grâce à un tableau de bord intelligent.

Caractéristiques clés :

- ✓ Architecture event-driven basée sur Kafka,
- ✓ Traitement intelligent via Spark Streaming,
- ✓ Couche d'adaptation hybride (règles + ML),
- ✓ Monitoring temps réel via Grafana/Prometheus,
- ✓ Stockage Lakehouse pour un accès performant.

Cette approche vise à dépasser les limites identifiées dans les projets similaires.

X. Méthodologie de recherche

Cette étude repose sur une analyse de la littérature scientifique existante, incluant des articles de recherche, des ouvrages spécialisés et des projets open-source. Les sources ont été sélectionnées à partir de bases académiques telles que Google Scholar et ResearchGate, ainsi que des documentations officielles des technologies étudiées. Cette approche a permis d'identifier les tendances actuelles, les limites des solutions existantes et les opportunités d'amélioration [21].

XI. Synthèse de la recherche

La revue des travaux et des projets existants montre que les pipelines modernes sont performants mais restent majoritairement statiques et peu autonomes. Les approches basées sur les règles, le ML et le RL ouvrent la voie à des pipelines capables d'apprendre et de s'ajuster intelligemment.

Notre contribution vise donc à concevoir un pipeline auto-adaptatif complet, intégrant des mécanismes avancés de détection, décision et adaptation. Cette recherche constitue la base théorique qui guidera la conception et la mise en œuvre de notre prototype dans la suite du PFE.

Références:

[1] G. Bell, T. Hey, A. Szalay, “Beyond the Data Deluge”, Science, 2009.

[2] Doug Laney, “3D Data Management: Controlling Data Volume, Velocity, and Variety”, Gartner.

[3] Kimball R., Ross M., “The Data Warehouse Toolkit”, Wiley.

[4] Vassiliadis P., “A Survey of ETL Technology”, ACM, 2009.

[5] Akidau T. et al., “The Dataflow Model”, Google, 2015.

[6] Armbrust M. et al., “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores”, VLDB, 2020.

[7] Marz N., Warren J., “Big Data: Principles and Best Practices of Scalable Realtime Data Systems”, Manning.

[8] Kreps J., “Questioning the Lambda Architecture”, O’Reilly.

[9] IBM, “Rule-Based Systems Overview”.

[10] Chandola V., Banerjee A., Kumar V., “Anomaly Detection: A Survey”, ACM.

[11] Sutton R., Barto A., “Reinforcement Learning: An Introduction”.

[12] Apache Kafka Documentation.

[13] Apache Spark Documentation.

[14] Apache Airflow Documentation.

[15] DZone, “Designing Scalable Real-Time Data Pipelines”, 2023.

[16] DZone, “Event-Driven Architectures for Data Streaming”, 2022.

[17] DZone, “Cloud-Native Data Pipelines on Kubernetes”, 2023.

[18] DZone, “Rule-Based vs Intelligent Adaptation in Data Pipelines”, 2023.

[19] Towards Data Science, “Machine Learning for Smart Data Pipelines”, 2024.

[20] Medium, “Reinforcement Learning for Adaptive Streaming Systems”, 2024.

[21] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016).

Site Reliability Engineering: How Google Runs Production Systems.

O’Reilly Media.